Augmented Reality Aircraft Tracer

Software Requirements Specification (SRS) Template v1.0

[Instructions Specific to the course SRS including formatting and any notable notation.]

This document is an annotated outline intended for specifying software requirements and is adapted from IEEE 29148-2018.

v1.0
Prepared By: Team F
Prepared For CSC380, SUNY Oswego
September 5th, 2024

Table of Contents

Table of Contents **2**

Change Log **3**

1. Introduction **3**

1.1 Purpose 4

1.2 Scope 4

1.3 Product Overview 4

1.3.1 Product Perspective 4

1.3.1.1 System Interfaces 4

1.3.1.2 User Interfaces 4

1.3.1.3 Hardware Interfaces 4

1.3.1.4 Software Interfaces 4

1.3.1.5 Communication Interfaces 5

1.3.1.6 Memory Constraints 5

1.3.1.7 Site Adaptation 5

1.3.1.8 Interfaces with Services 5

1.3.2 Product Functions 5

1.3.3 User characteristics 5

1.3.4 Limitations 5

1.4 Definitions 5

2. References **6**

3. Requirements **6**

3.1 Functions 7

3.2 Performance Requirements 7

3.3 Usability Requirements 7

3.4 Interface Requirements 8

3.5 Logical Database Requirements 8

3.6 Design Constraints 8

3.7 Software System Attributes 8

3.8 Supporting Information 9

4. Verification **9**

4.1 Functions 9

4.2 Performance Requirements 9

4.3 Usability Requirements 9

4.4 Interface Requirements 9
4.5 Logical Database Requirements 9
4.6 Design Constraints 9
4.7 Software System Attributes 10
4.8 Supporting Information 10

5. Appendix A – Tailoring Policies **10**
5.1 Assumptions and Dependencies 10
5.2 Acronyms and Abbreviations 10
5.3 Tailoring Policies 11

6. Appendix B –Copyright **11**
6.1 Author Names 11
6.2 Creative Commons License 11

Change Log

Name	Date	Reason For Changes	Version

1. Introduction

[This section of the document is intended to give a wide overview of the project, not identify specific solutions. If a section does not apply, ***do not remove it from the document,*** *instead, write a brief note on why the section does not apply to the SUD.]*

1.1 Purpose

[Identify the primary reason for the SRS, the system, and the primary audience of the SRS and the system, if they are different. Keep it as short as possible, you will elaborate in future sections.]

The primary purpose of this SRS document is to define the requirements needed for developing a tool that incorporates real-time flight data into the X-Plane flight simulator, providing a simplified environment for testing Traffic Collision Avoidance Systems. The primary audience of this document and system includes the development team, project stakeholders, and students at the State University of New York at Oswego.

1.2 Scope

[Identify the scope of the SRS document, what will be covered, what will be excluded? Additionally, identify the general scope of the system you are building. Keep these brief, use bullet points to help organize the information.]

Scope of the SRS:

- Define functional and non-functional requirements for the system.
- Determine interfaces between our plugin and OpenSky Network, X-Plane, and AirborneCPS.
- Possible add-ons of improved interpolation and a 2D flight path map.
- Outline performance, usability, and other system attributes.

Scope of the System:

- Inclusions:
    - Development of a plugin for X-Plane.
    - Acquisition of real-time ADS-B data from OpenSky Network.
    - Parsing and interpolation of ADS-B data for showing real-time flights in X-Plane.
    - Rendering aircraft into X-Plane’s simulated environment.
    - UDP output stream based on user location to AirborneCPS for TCAS integration.
    - Creation of a database to store flight paths for offline use.
    - 2D map application for displaying real-world flight paths.
- Exclusions:
    - Development of ADS-B data acquisition tools.
    - Modification of X-Plane’s base functionality.
    - Development or modification of a TCAS.

1.3 Product Overview

1.3.1 Product Perspective

[Identify the product's relationships with other products already in existence or currently in development. If this product is part of a larger system, identify that here and notate how the system will interact with external actors.]

This system will be similar to other real-time augmented reality aircraft simulations that render real-world aircraft in their simulation. It will function as an extension of the X-Plane flight simulator, using external APIs for realism. It will interact with existing systems, namely OpenSky Network for data input, X-Plane for rendering aircraft, and AirborneCPS for collision avoidance.

1.3.1.1 System Interfaces

[List the system interfaces and identify the functionality of that system. How will it be used in conjunction with the system you are developing?]

- OpenSky Network
  - Used for gathering real-time ADS-B data, including ICAO, position coordinates, velocity, altitude, and track.
  - Input for rendering aircraft in X-Plane and displaying flight paths in the 2D map application.
- X-Plane
  - Flight simulation software that will be used for aircraft rendering.
  - Supports add-ons that interface with its internal systems.
- AirborneCPS
  - A system that will receive and process UDP streams of interpolated aircraft data for TCAS implementation.

1.3.1.2 User Interfaces

[List the characteristics of each interface between the product and its users. Include the modes of operation in the user organization, data processing support functions, and any backup or data recovery operations present in the system.]

- X-Plane
  - The primary user interface for the system. Users can control their own aircraft, view surrounding representations of real-world

aircraft, and interact with other aircraft through TCAS implementation.

- 2D Map Application
    - Users can interact with a separate 2D map displaying flight paths based on interpolated ADS-B data, zooming in and out to view aircraft flight paths at different levels.
- The plugin will simply be switched on by the user, and will not have any ability to be configured by the users.

1.3.1.3 Hardware Interfaces

[Specify the logical characteristics of each interface between the software product and the hardware components of the system. This includes configuration characteristics. It also covers such matters as what devices are to be supported, how they are to be supported, and any relevant protocols.]

The system shall be compatible with standard machines that meet X-Plane requirements. No additional hardware is necessary.

1.3.1.4 Software Interfaces

[List the name, mnemonic, specification number, version number, and source of each software product, be sure to include the purpose of each software product you are integrating and how it will be incorporated into the SUD.]

- OpenSky Network API
    - Mnemonic: OpenSky Network
    - Version: Version 1.4
    - Source: https://openskynetwork.github.io/opensky-api/
    - Purpose: The OpenSky Network API provides real-time ADS-B data, including ICAO, altitude, latitude, longitude, and track.
    - Incorporation: The system will interact with the OpenSky Network API to receive real-time ADS-B data that will be used as the primary input for rendering aircraft.
- X-Plane Flight Simulation Software
    - Mnemonic: X-Plane
    - Version: Version 12.1.2
    - Source: https://www.x-plane.com/
    - Purpose: X-Plane provides users with an environment to simulate flight operations and aircraft behavior.

    - Incorporation: The system will communicate with X-Plane to render aircraft within its simulated environment based on interpolated ADS-B data.

1.3.1.5 Communication Interfaces

[Identify the various networking protocols and other such communication interfaces. Unique and non-standard protocols should be documented here, standard protocols should be referenced and cited here and in the reference document section]

- The system will use UDP for sending interpolated aircraft data to AirborneCPS.
- The system will use HTTPS for making requests for ADS-B Data to OpenSky Network.

1.3.1.6 Memory Constraints

[List any limitations on primary and secondary memory, as well as any applicable characteristics.]

The minimum requirements for X-Plane:

- Disk Space: 25GB
- CPU: Intel Core i3, i5, i7, i9 CPU with 4 or more cores, or AMD Ryzen 3, 5, 7, 9.
- Memory: 8GB RAM
- Video Card: a Vulkan 1.3-capable video card from NVIDIA or AMD with at least 2 GB VRAM

1.3.1.7 Site Adaptation

[Identify any situations, initializations, or protocols that are specific to a give site, mission, or operational model. If you are tailoring a product to meet specific client needs that differ from normal operation, include it in this section.]

- The system will communicate with OpenSky Network's REST API to obtain all states of all aircraft in the world every ten seconds to avoid credit exhaustion.

1.3.1.8 Interfaces with Services

[List any interaction with any SAAS (Software as a Service) or cloud-based services.]

- Interaction with OpenSky Network as a cloud-based service for ADS-B data retrieval.

1.3.2 Product Functions

[This section of the document is intended to describe all of the major functions the product will be able to perform. Functions should be organized in such a way that the list of functions is understandable to anyone who is reading through the document for the first time.]

- Continuously acquire real-time ADS-B data from OpenSky Network every twenty seconds.
- Parse the acquired ADS-B data for relevant information, namely ICAO, latitude, longitude, altitude, velocity, and heading.
- Interpolate between aircraft data points to ensure smooth aircraft movement.
- Load database containing flight paths for offline plugin use.
- Get the user's in-game geographical location from X-Plane.
- Output interpolated aircraft data via UDP stream for integration with AirborneCPS based on user location.
- Display stored flight paths on a separate 2D map application.

1.3.3 User characteristics

[Describe the characteristics of the product's intended group of users. Include things such as technical expertise, and any information that may impact usability or accessibility to the product.]

The primary users should be familiar with X-Plane's base functions and controls and have prior knowledge about Traffic Collision Avoidance Systems.

1.3.4 Limitations

[Identify any limitations that will impact the development. Include things such as hardware limitations, safety and security considerations, quality requirements, regulatory requirements and/or policies, etc.]

- Requires a compatible version of X-Plane and adequate hardware to run both the simulator and plugin efficiently.
- Limitations on the amount and availability of ADS-B data able to be received within a day.
- Real time exact positioning is not the goal of the plugin, rather smooth rendering.

1.4 Definitions

[Include any terms and definitions needed to understand the SRS or the System Under Development (SUD), terms placed here should also be placed in the Appendix. Terms should use the full name and the general definition of that term, any abbreviations that will be used in the document and the source should be placed in Appendix 5.2]

Term	Definition
Example: *context of use*	*users, tasks, equipment (hardware, software, and materials), and the physical and social environments in which a product is used*
System Under Development	The system that is actively being developed.
Automatic Dependent Surveillance-Broadcast	Surveillance technology that allows aircraft to convey flight information to air traffic controllers and other aircraft.
User Datagram Protocol	A protocol used to send packets across networks without establishing a connection. Fast but packet loss is possible.
Representational State Transfer	An architectural style for designing APIs.
Traffic Collision Avoidance System	A safety system that monitors the airspace around an aircraft and provides guidance on how to avoid collisions.

2. References

[Include citations to external sources and resources in this section. References to other internal documents can be placed here but should also be referenced in the appendix.
Example: ISO/IEC/IEEE 29148.2018, Systems and software engineering
Life cycle processes — Requirements engineering]

- *Matthias Schäfer, Martin Strohmeier, Vincent Lenders, Ivan Martinovic* ***and*** *Matthias Wilhelm. "Bringing Up OpenSky: A Large-scale ADS-B Sensor Network for Research". In Proceedings of the 13th IEEE/ACM International Symposium on Information Processing* ***in*** *Sensor Networks (IPSN), pages 83-94, April 2014.*
- *Meyer, Austin. "X-Plane 12". X-Plane 12, 13 Mar. 2024, www.x-plane.com/.*

- *The OpenSky Network. "The OpenSky Network API documentation". 2021, https://openskynetwork.github.io/opensky-api/.*
- *"Automatic Dependent Surveillance - Broadcast (ADS-B)." Automatic Dependent Surveillance - Broadcast (ADS-B) | Federal Aviation Administration, www.faa.gov/about/office_org/headquarters_offices/avs/offices/afx/afs/afs400/afs410/ads-b. Accessed 6 Dec. 2024.*

3. Requirements

[This section should contain all the software requirements at a level of detail sufficient enough to enable designers to design a system, and for testers to test that system, in a way that satisfies the requirements. Each requirement should be perceivable by users, operators, or other external systems. At minimum, the description should include the inputs and outputs of the system, and all functions performed by the system in response to an input or in support of an output. Specific requirements should include the following characteristics:

- *Correct*
- *Unambiguous*
- *Complete*
- *Consistent*
- *Verifiable*
- *Modifiable*
- *Traceable*
- *Ranked for importance and/or stability*
- *uniquely identifiable (usually by numbering)*
- *organized in a way that allows for maximum readability*

The purpose of the requirement is not to dictate design, but rather to guide designers to make the safest, most correct version of the system possible. Do not attempt to build solutions to your written requirements]

3.1 Functions

[Define the fundamental actions that the system must take in order to accept inputs and generate outputs. It may make sense to organize or partition the functional requirements into sub-functions or sub-processes, do not expect development to mimic this organization. Functional requirements are typically identified using "shall" statements and some functions that should be included in this section are:

- The system shall verify that the received ADS-B data conforms to the expected format and contains all required information.

- The system shall parse the acquired ADS-B data, and then it shall store the relevant ADS-B data.
- The system shall interpolate the relevant aircraft data, and then it shall render the aircraft into X-Plane.
- The system shall get the user's location, and then it shall send the interpolated aircraft data via UDP stream to AirborneCPS based on the user's location.
- If the system experiences data overflow, it shall temporarily stop accepting new ADS-B data.
- The system shall include a database of flight paths to sample.
- If the system experiences data loss, it shall continue moving aircraft on the path they are currently on.
- If an internet connection is not available at any point while running the plugin, it shall pull stored ADS-B data from the flight path database to render aircraft and send UDP streams.

3.2 Performance Requirements

[In measurable terms, specify the numerical requirements of the system. Include static performance requirements such as the number of terminals, simultaneous users, etc. As well as dynamic performance requirements such as the number of tasks able to be completed in a set period of time.]

- The system shall operate on a single machine running X-Plane with the plugin for one user.
- The system shall process and render data updates at a smooth rate.
- The system shall handle the rendering of all aircraft without significant performance loss.
- The system shall be capable of providing functionality under connection loss.

3.3 Usability Requirements

[Define usability and quality requirements that are measurable in effectiveness, efficiency, satisfaction, and in avoidance of harm that could arise from specific use cases.]

- The system shall show aircraft in X-Plane such that they reflect real-world aircraft patterns.
- The system shall include a separate application to display flight paths.

3.4 Interface Requirements

[List all inputs and outputs from the system. It should mirror but not repeat the information found in sections 4.2- 4.6. For each defined interface, be sure to include:

- *the name of the item*

- *description of the purpose of the interface*
- *source of input OR output destination*
- *range, accuracy, and/or tolerance*
- *units of measurement*
- *timing***
- *I/O relationships*
- *data formats*
- *command formats*

any information included within the I/O.]

- The plugin must access several data points provided by the OpenSkyNetwork API:

icao24	string	One or more ICAO24 transponder addresses are represented by a hex string (e.g. abc9f3). To filter multiple ICAO24 append the property once for each address. If omitted, the state vectors of all aircraft are returned.
longitude	float	WGS-84 longitude in decimal degrees. Can be null.
latitude	float	WGS-84 latitude in decimal degrees. Can be null.
baro_altitude	float	Barometric altitude in meters. Can be null.
velocity	float	Velocity over ground in m/s. Can be null.

- The plugin shall connect to the Open Sky Network API to retrieve longitude, latitude, barometric altitude, velocity, and heading. These states shall be used to place an aircraft object within the X-Plane environment and to calculate the interpolation endpoints.
- The plugin must use UDP to broadcast several data points based on the interpolation data created. These include longitude and latitude, geo_altitude, velocity, vertical_rate.

3.5 Logical Database Requirements

[Identify the logical requirements for information that will be placed in a database. These requirements should consider and include the following:

- *Types of information that will be used.*
- *Frequency of use*
- *accessibility*
- *integrity constraints*
- *security*
- *data retention*
- *data entities and their relationships.]*

- The plugin shall use a generated database that stores the flight paths of all aircraft in the world for a 24-hour period.
- The system shall further filter the database to only contain specific aircraft.

3.6 Design Constraints

[List any constraints on the system. These constraints should be from external sources such as regulatory standards, and legal, or project limitations.]

- The system must be compatible with X-Plane 12.
- The system must operate within the performance constraints of the hardware specified for X-Plane.
- The system must be completed by December 6th, 2024.
- The system's first implementation must be completed by November 1st, 2024

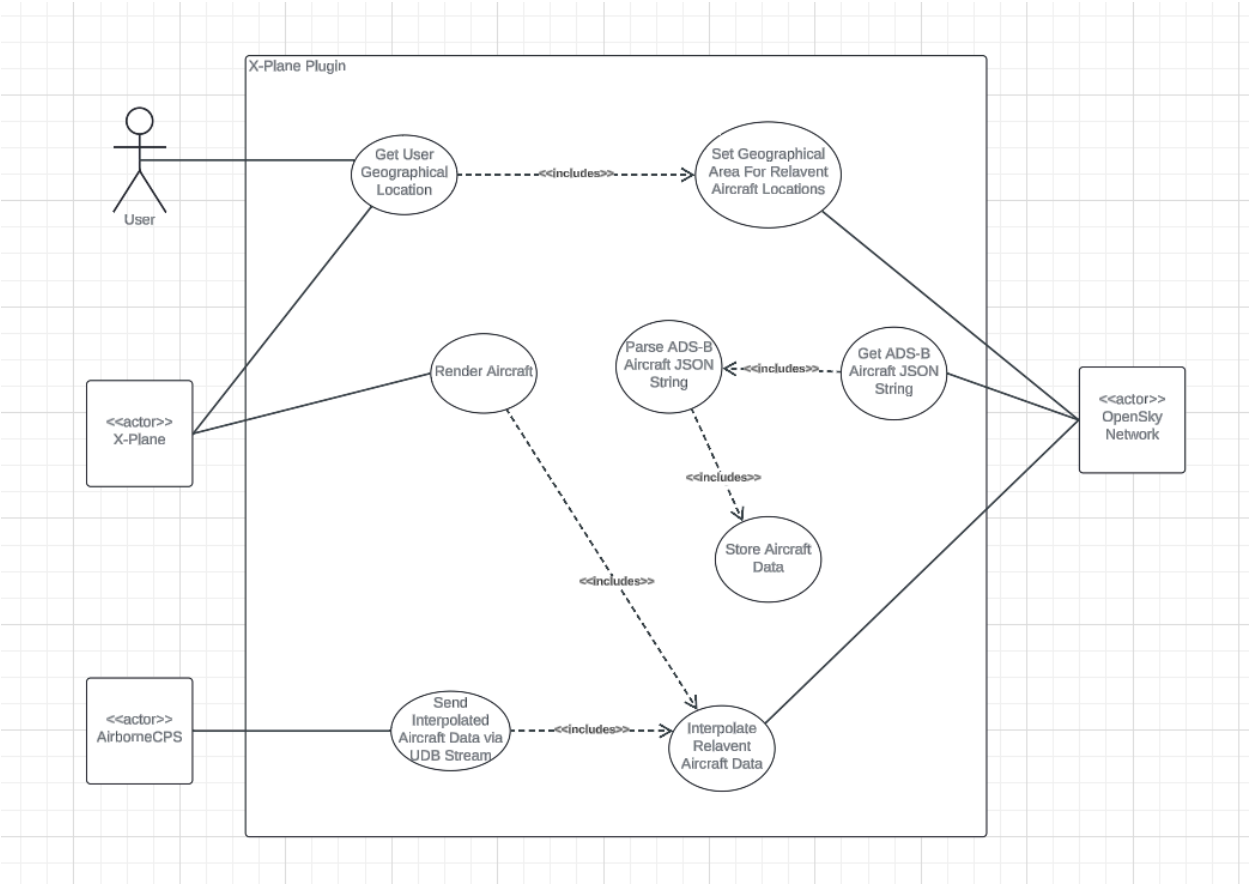
3.7 Software System Attributes

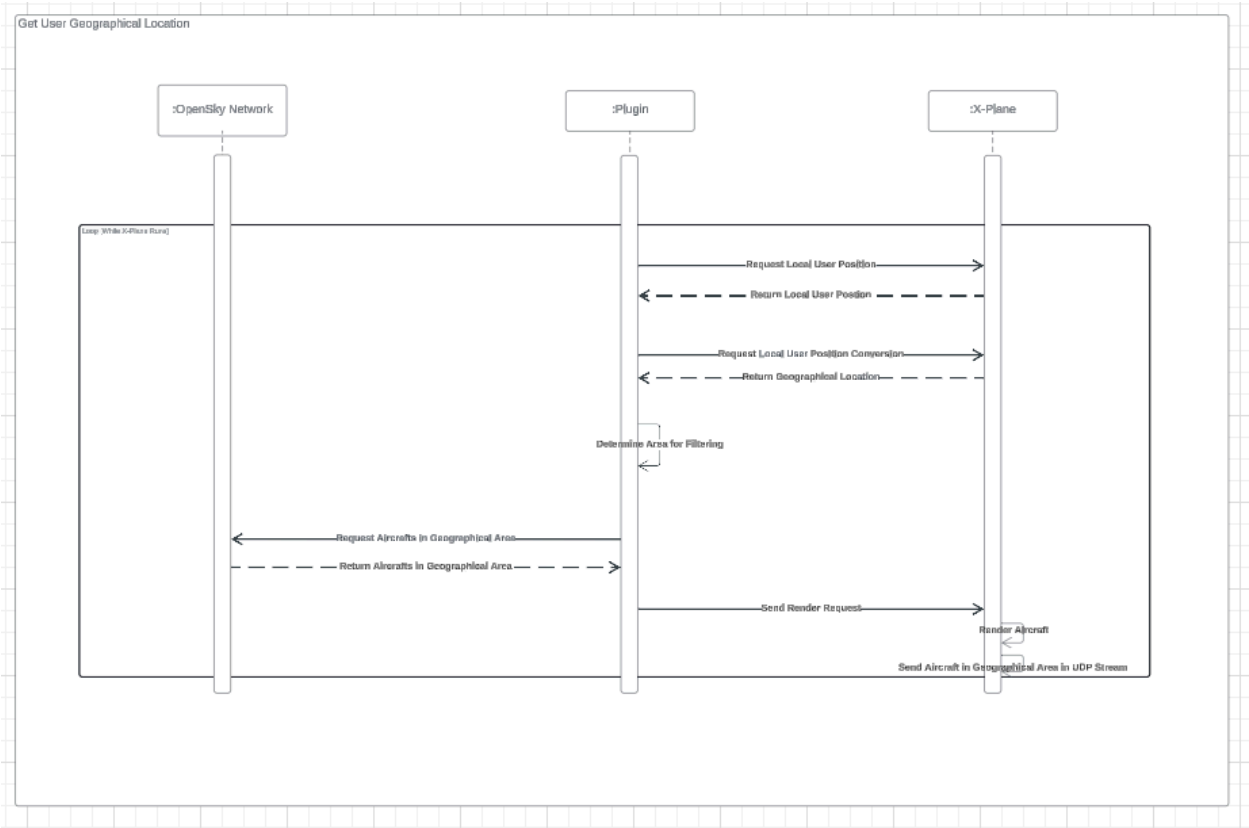
[For each of the attributes of the software system (Reliability, Availability, Security, Maintainability, Portability, etc.), list the factors that will establish functionality or stability. For example, when establishing requirements for the Security attribute, you may include one that restricts communication between two areas of the program and another.]

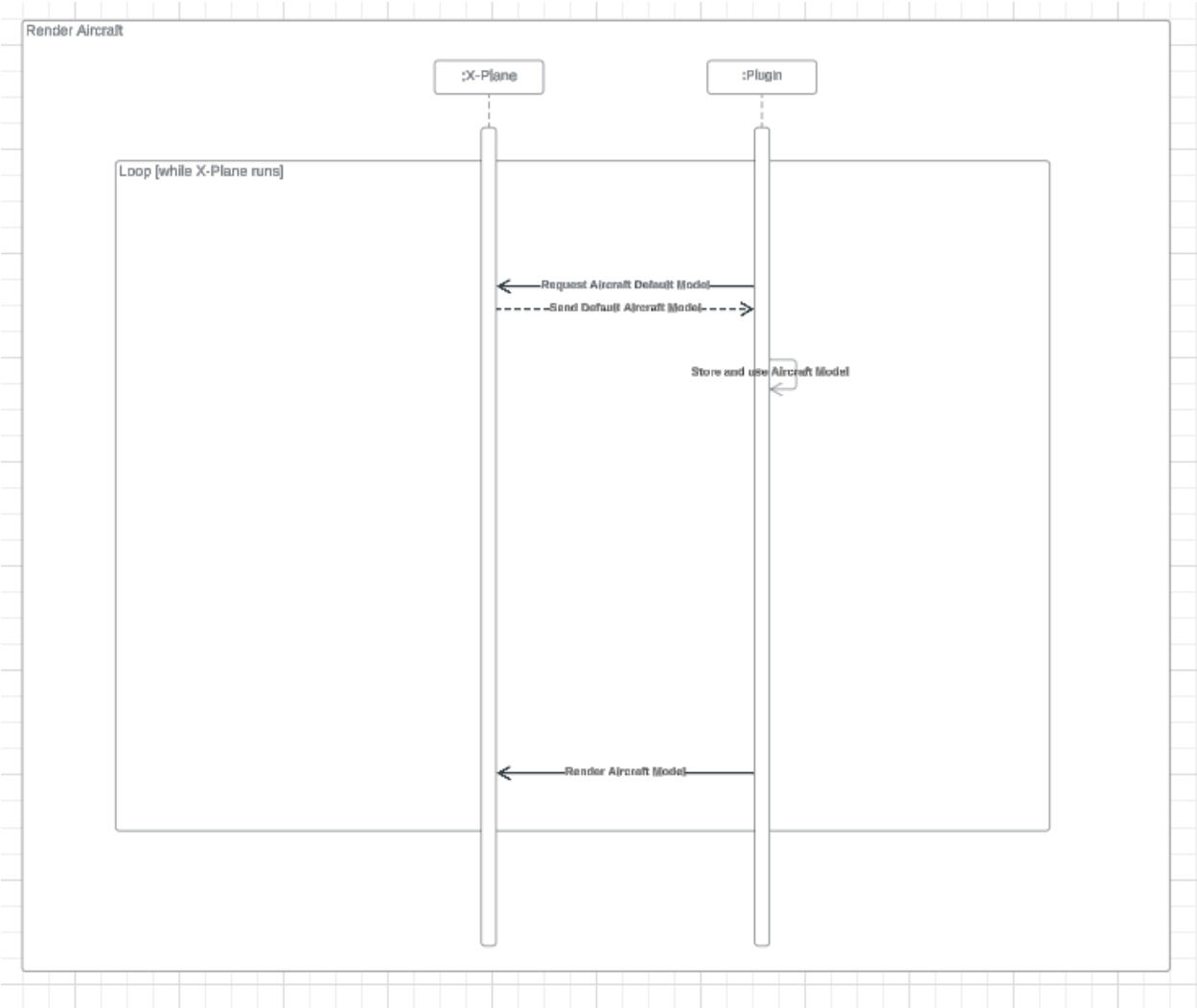
- Reliability: The system shall maintain data integrity by using stored data when the internet connection is not available upon starting.
- Maintainability: The system shall provide documentation to support future maintenance.
- Usability: The system shall include help documentation to aid users in understanding its functionality.
- Scalability: The system shall be able to handle an increase in aircraft rendering without performance loss.

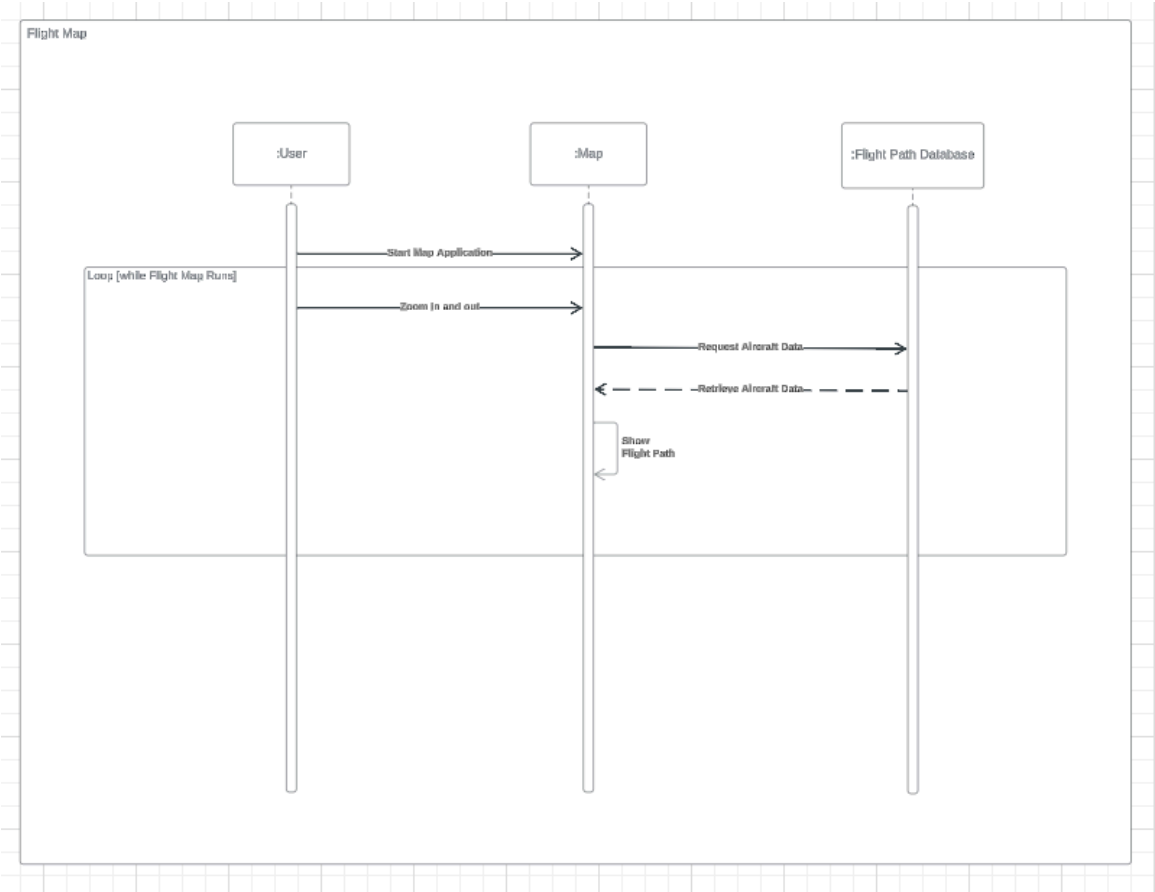
3.8 Supporting Information

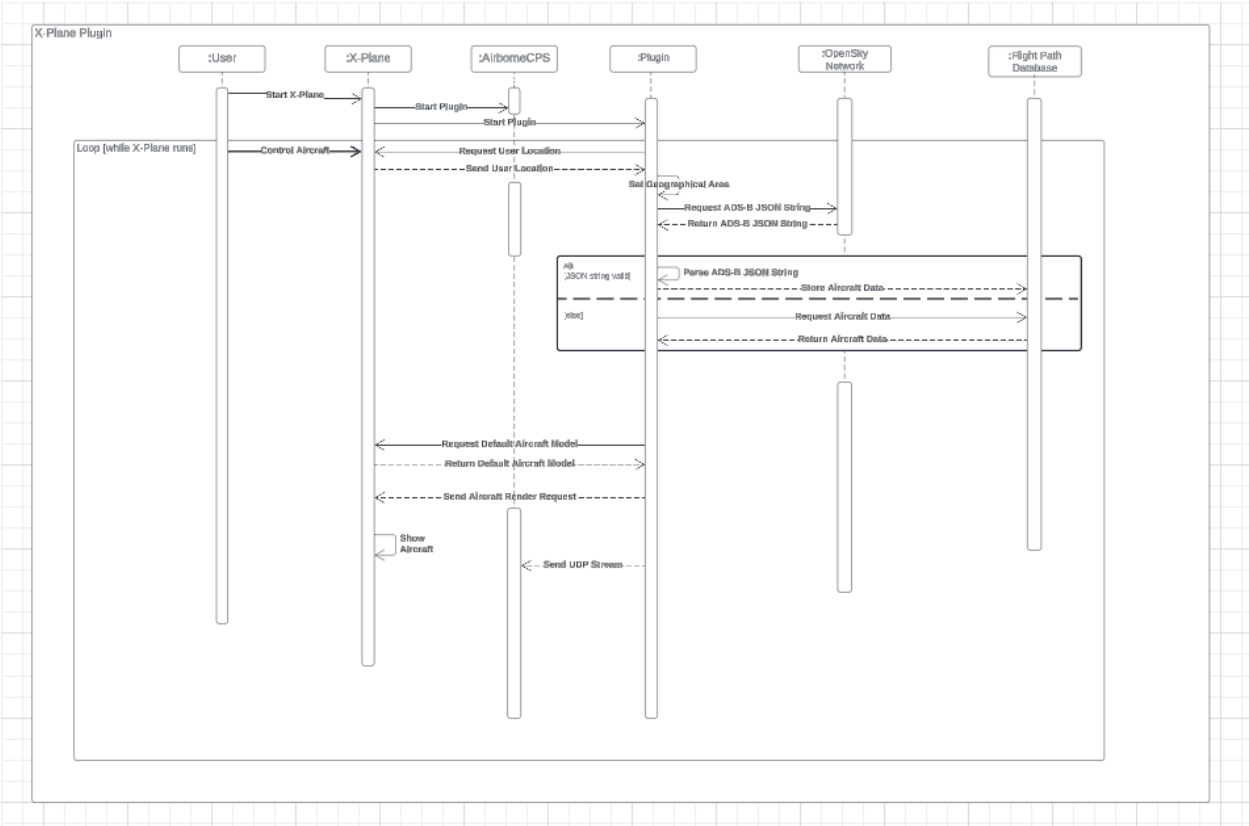
[Add any additional information needed to understand the SRS, including background information, problem descriptions, packaging instructions for code, sample input/output formats, etc.]

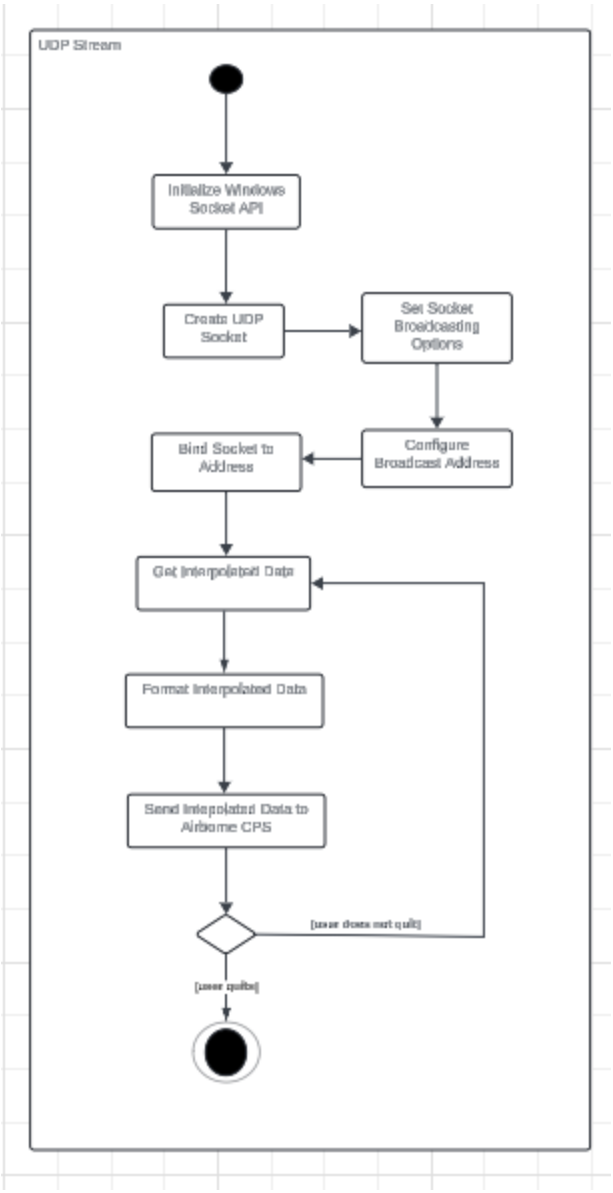

X-Plane Plugin
User
Get User Geographical Location
<<includes>>
Set Geographical Area For Relavent Aircraft Locations
<<actor>> X-Plane
Render Aircraft
Parse ADS-B Aircraft JSON String
<<includes>>
Get ADS-B Aircraft JSON String
<<actor>> OpenSky Network
<<includes>>
Store Aircraft Data
<<includes>>
<<actor>> AirborneCPS
Send Interpolated Aircraft Data via UDB Stream
<<includes>>
Interpolate Relavent Aircraft Data

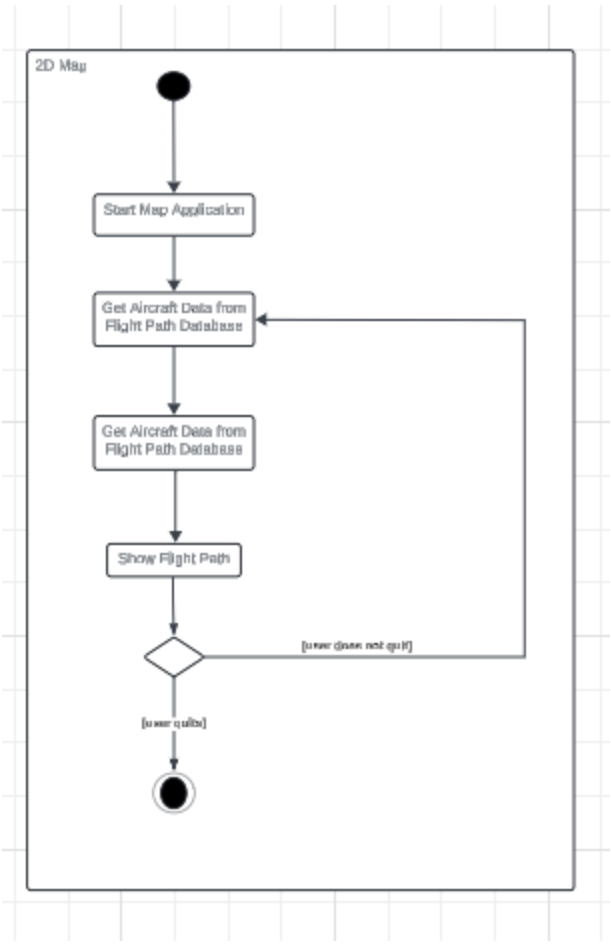

Get User Geographical Location
:OpenSky Network
:Plugin
:X-Plane
Loop [While X-Plane Runs]
Request Local User Position
Return Local User Posdion
Request Local User Position Conversion
Return Geographical Location
Determine Area for Filtering
Request Aircrafts in Geographical Area
Return Aircrafts in Geographical Area
Send Render Request
Render Aircraft
Send Aircraft in Geographical Area in UDP Stream

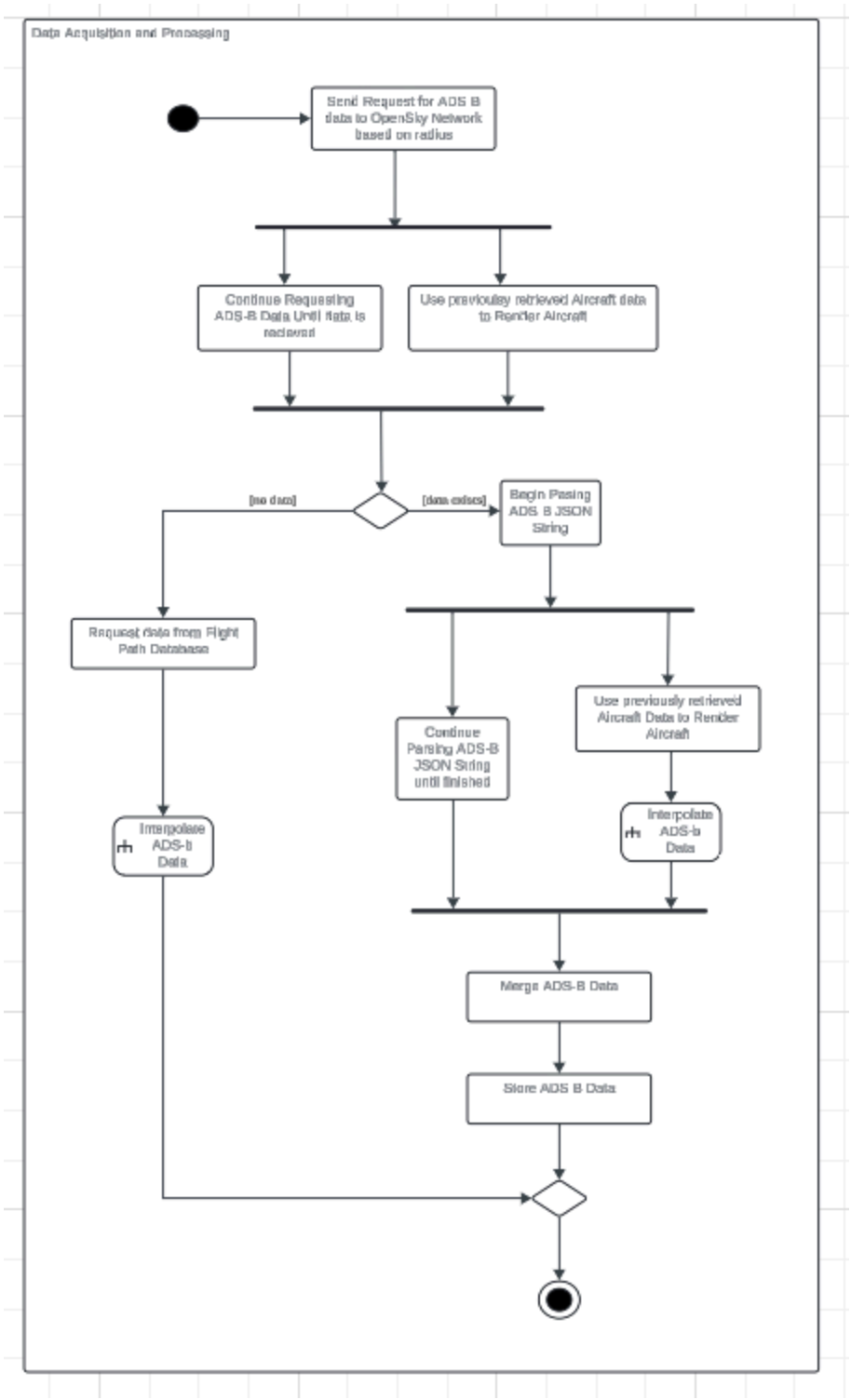

Render Aircraft
:X-Plane
:Plugin
Loop [while X-Plane runs]
Request Aircraft Default Model
Send Default Aircraft Model
Store and use Aircraft Model
Render Aircraft Model

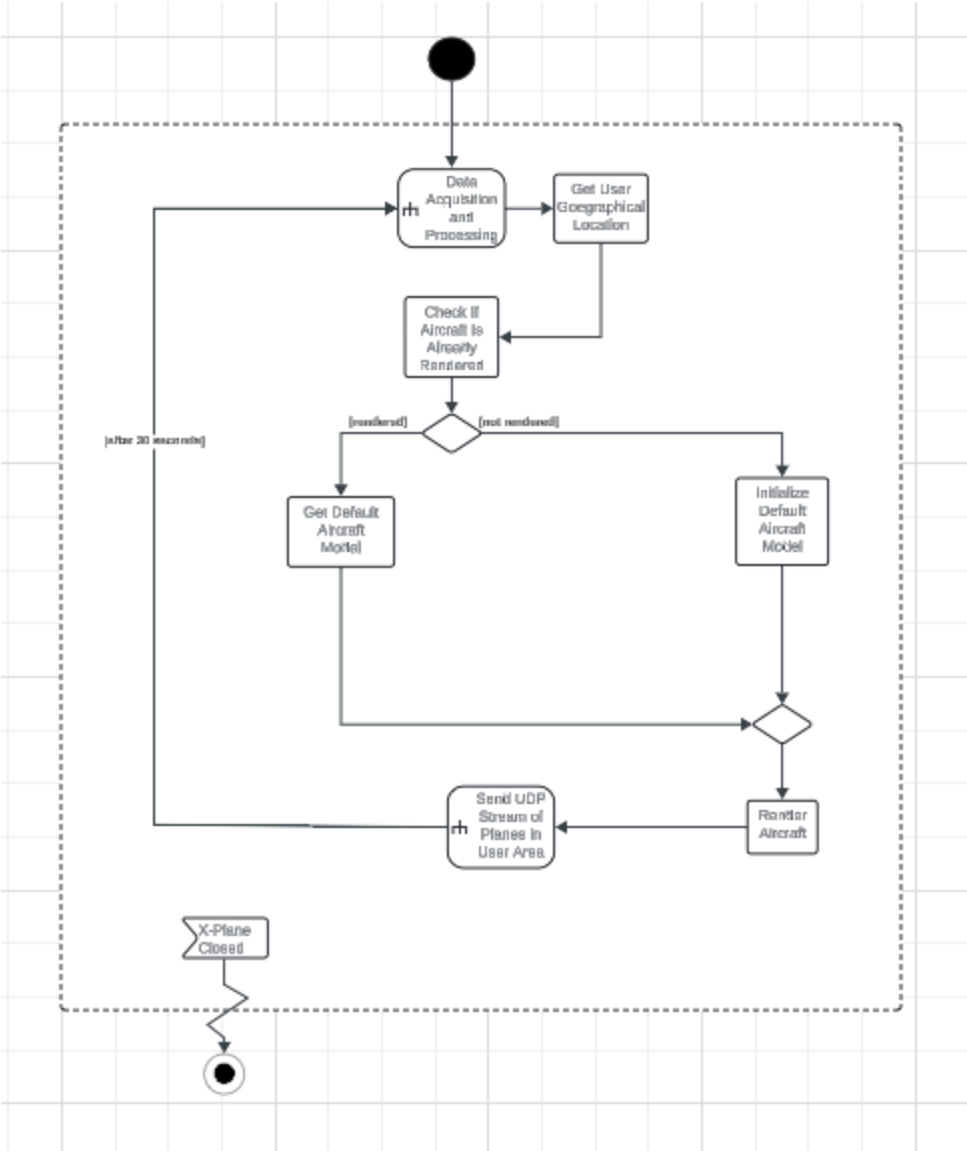

Flight Map
:User
:Map
:Flight Path Database
Start Map Application
Loop [while Flight Map Runs]
Zoom In and out
Request Aircraft Data
Retrieve Aircraft Data
Show
Flight Path

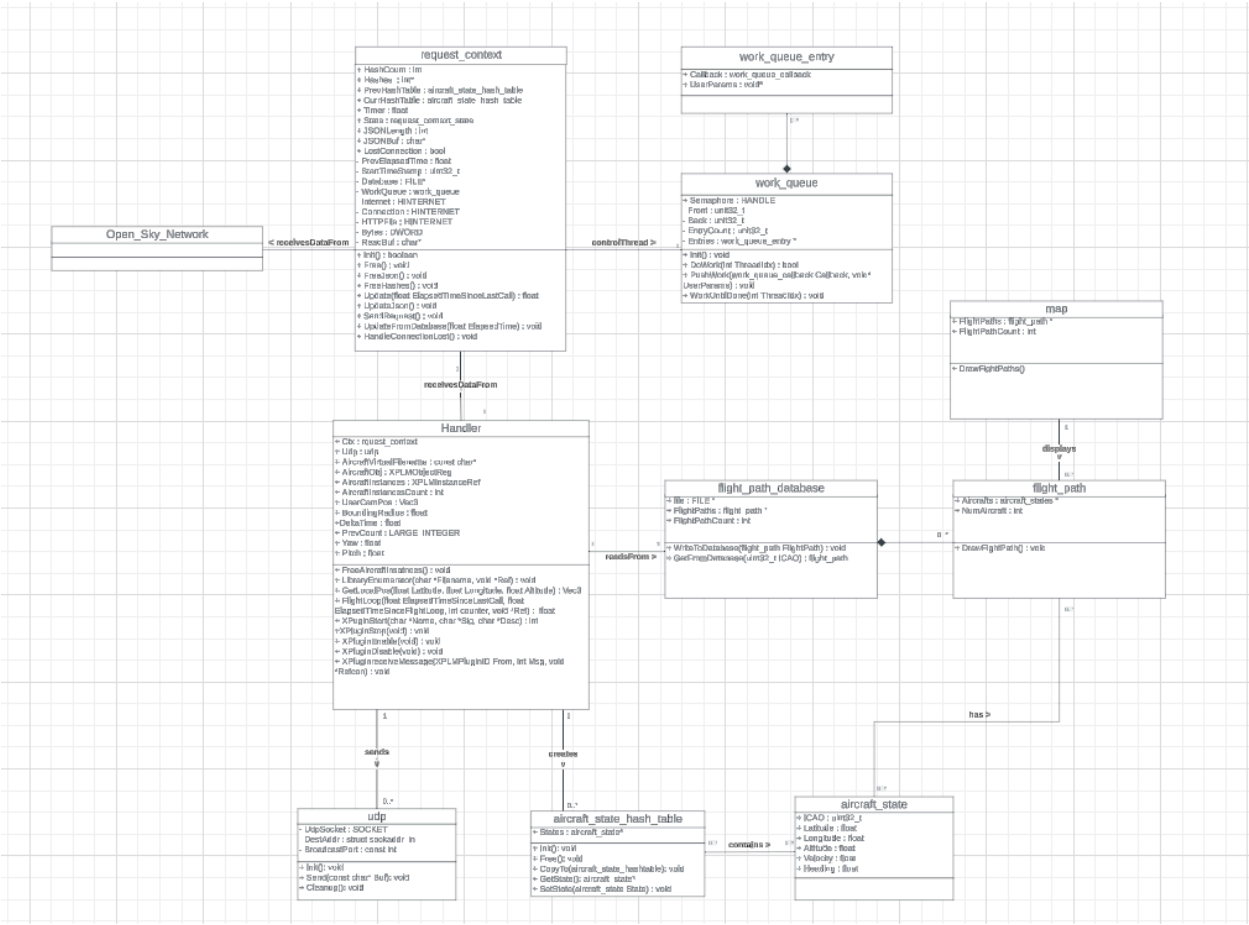

X-Plane Plugin
:User
:X-Plane
:AirborneCPS
:Plugin
:OpenSky Network
:Flight Path Database
Start X-Plane
Start Plugin
Start Plugin
Loop [while X-Plane runs]
Control Aircraft
Request User Location
Send User Location
Set Geographical Area
Request ADS-B JSON String
Return ADS-B JSON String
Alt
[JSON string valid]
Parse ADS-B JSON String
Store Aircraft Data
[else]
Request Aircraft Data
Return Aircraft Data
Request Default Aircraft Model
Return Default Aircraft Model
Send Aircraft Render Request
Show Aircraft
Send UDP Stream


UDP Stream
Initialize Windows Socket API
Create UDP Socket
Set Socket Broadcasting Options
Configure Broadcast Address
Bind Socket to Address
Get Interpolated Data
Format Interpolated Data
Send Interpolated Data to Airborne CPS
[user does not quit]
[user quits]


2D Map
Start Map Application
Get Aircraft Data from Flight Path Database
Get Aircraft Data from Flight Path Database
Show Flight Path
[user does not quit]
[user quits]


Data Acquisition and Processing
Send Request for ADS B data to OpenSky Network based on radius
Continue Requesting ADS-B Data Until data is recieved
Use previously retrieved Aircraft data to Render Aircraft
[no data]
[data exists]
Begin Pasing ADS B JSON String
Request data from Flight Path Database
Continue Parsing ADS-B JSON String until finished
Use previously retrieved Aircraft Data to Render Aircraft
Interpolate ADS-b Data
Interpolate ADS-b Data
Merge ADS-B Data
Store ADS B Data


Data Acquisition and Processing
Get User Geographical Location
Check If Aircraft Is Already Rendered
[rendered]
[not rendered]
[after 30 seconds]
Get Default Aircraft Model
Initialize Default Aircraft Model
Send UDP Stream of Planes in User Area
Render Aircraft
X-Plane Closed


request_context
work_queue_entry
work_queue
Open_Sky_Network
map
Handler
flight_path_database
flight_path
udp
aircraft_state_hash_table
aircraft_state
receivesDataFrom
controlThread
receivesDataFrom
displays
roadsFrom
has
sends
creates
contains

4. Verification

[List all inputs and outputs from the system. It should mirror but not repeat the information found in sections 3.1- 3.8. For each defined interface, be sure to include:

- *the name of the item*
- *description of the purpose of the interface*
- *source of input OR output destination*
- *range, accuracy, and/or tolerance*
- *units of measurement*
- *timing*
- *I/O relationships*
- *data formats*
- *command formats*
- *any information included within the I/O.]*

4.1 Functions

[See sections 4.0, and 3.1 for specific directions about what outputs should be included here.]

- The system shall request the user's geographic location from X-Plane, and then it shall generate an area with a radius of about 10 nautical miles centered on that location to request ADS-B data from OpenSky Network.
- The system shall request ADS-B JSON Strings from OpenSky Network every twenty seconds, and then it shall verify that the string is a JSON string, indicating the data is valid.
- The system shall parse the JSON String for each aircraft's ICAO, latitude, longitude, altitude, velocity, and heading, and then it shall interpolate the parsed data at a target rate of 30 frames per second.
- The system shall store all of the data in a hash table hashed by the aircraft's ICAO, and then it will store the ICAO into an array.
- The system shall iterate over all the ICAOs stored in the ICAO array and index into the two hash tables and retrieve the aircraft's geographical location at 2 points in time and linearly interpolate the two based on the accumulated seconds since the program has started.
- To interpolate latitude, longitude, and altitude, the system shall use the linear interpolation formula $lerp(A, B, t) = (1 - t) * A + t * B$, where A is the previous point, B is the current point, and t is the timer over a send time threshold of twenty seconds.
- The system shall use the interpolated ADS-B data to render the aircraft into X-Plane.

- The system shall request the user's geographical location from X-Plane and then shall send a UDP stream of the interpolated ADS-B data to AirborneCPS for all of the planes within ten nautical miles of the user.
- The system shall loop and repeat, starting from retrieving the ADS-B data from the OpenSky Network.
- The system shall pull flight path data from the database when there is an absence of internet connection upon starting X-Plane.
- If the system receives invalid or no data from OpenSky Network or loses internet connection, it shall interpolate flight paths from the database.
- The system shall have a 2D Map that displays flight paths from the flight path database.

4.2 Performance Requirements

[See sections 4.0, and 3.2 for specific directions about what outputs should be included here.]

- The plugin shall process data updates at the minimum target framerate X-Plane requests which at default is 50 frames per second.
- While rendering plugin aircraft, X-Plane itself shall run at atleast 30 frames per second.

4.3 Usability Requirements

[See sections 4.0, 3.3 for specific directions about what outputs should be included here.]

- The system shall display pitch and heading accurately to reflect real world aircraft.
- The system shall come with a separate 2D map application to visualize typical flight paths in the demo area for X-Plane of Portland, Oregon.

4.4 Interface Requirements

[See sections 4.0, 3.4 for specific directions about what outputs should be included here.]

- Aircraft Data Input Interface
    - Name: ADS-B Data
    - Purpose: Receives real-time ADS-B data from the OpenSky Network API
    - Units of measurement:
        - Latitude/Longitude: Decimal Degrees
        - Altitude: Meters
        - Velocity: Meters/Second
        - Heading: Decimal Degrees
    - Timing: ADS-B updates every 10 seconds
    - I/O Relationship: Input for parsing, interpolating, and rendering aircraft into X-Plane

    - Data Format: JSON String
    - Includes ICAO, timestamp, latitude, longitude, altitude, and track

4.5 Logical Database Requirements

[See sections 4.0, 3.5 for specific directions about what outputs should be included here.]

- The plugin shall use a generated database that stores the flight paths of all aircraft in the world for a 24-hour period
- Flight Path Database
    - Types of Information: ICAO, latitude, longitude, altitude, velocity, and heading.
    - Frequency of Use: Used when the system is launched without an internet connection, and for creating the 2D map.
    - Accessibility: Accessible by the plugin.
    - Integrity Constraints: Ensure that all fields are complete.
    - Data Retention: Ran a separate application for 24-hours to populate the database with all flights. Then ran another application to filter the database based on the criteria of each aircraft containing at least 5,000 points and one of the points must be in Oregon.
    - Relationships: Each flight record has ICAO, latitude, longitude, altitude, velocity, and heading. One-to-many relationship: each aircraft can have many flight records.

4.6 Design Constraints

[See sections 4.0, 3.6 for specific directions about what outputs should be included here.]

- The plugin shall not have live updates.
- The plugin shall be latent by approximately twenty seconds.
- The plugin shall not simulate more than the given flights in about 10 nautical miles.

4.7 Software System Attributes

[For each of the attributes of the software system (Reliability, Availability, Security, Maintainability, Portability, etc.), list the factors that will establish functionality or stability. For example, when establishing requirements for the Security attribute, you may include one that restricts communication between two areas of the program and another.]

[See sections 4.0, 3.7 for specific directions about what outputs should be included here.]

- Reliability: The system shall maintain data integrity by getting stored ADS-B data from the Flight Path Database if no internet connection is available at the start of the plugin.
- Availability: The plugin shall be available to anyone with the files to install it.
- Maintainability: The source code shall be written in a readable manner with proper documentation to support future maintenance.
- Portability: The system shall run on any system that supports X-Plane.

4.8 Supporting Information

[See sections 4.0, 3.8 for specific directions about what outputs should be included here.]

5. Appendix A – Tailoring Policies

5.1 Assumptions and dependencies

[Identify any and all factors that may impact the implementation and execution of the requirements written below. These factors do not add a constraint but may impact development if they are changed. Example: a major update to an operating system(OS) on which the SUD is intended to run impacts the implementation of one of the core features. The version of the OS that the system had intended to run on should be listed in this section.]

5.2 Acronyms and Abbreviations

Term	Definition	Abbreviation	Source (if applicable)
Example: context of use	*users, tasks, equipment (hardware, software and materials), and the physical and social environments in which a product is used*	*COU*	*[SOURCE: ISO/IEC 25000:2014, 4.2]*
System Under Development	The system which is actively being developed.	SUD	
Automatic Dependent Surveillance-Broadcast	An advanced surveillance technology that combines an aircraft's positioning source, aircraft avionics, and a ground infrastructure to	ADS-B	*[Source: "Automatic Dependent Surveillance - Broadcast (ADS-B)." Automatic Dependent Surveillance - Broadcast (ADS-B) Federal Aviation Administration, www.faa.gov/about/office_org/he*

	create an accurate surveillance interface between aircraft and ATC		*adquarters_offices/avs/offices/afx/afs/afs400/afs410/ads-b.]*
Air Tower Control	The process or system by which the movements of aircraft are monitored and directed by ground personnel communicating with pilots by radio	ATC	*[SOURCE:"Air Traffic Control Definition & Meaning." Merriam-Webster, Merriam-Webster, www.merriam-webster.com/dictionary/air%20traffic%20control. Accessed 6 Dec. 2024.]*
User Datagram Protocol	A transport layer communication protocol, UDP is a very common protocol for voice and video traffic.	UDP	*[SOURCE: What Is the User Datagram Protocol (UDP)? | Cloudflare, www.cloudflare.com/learning/ddos/glossary/user-datagram-protocol-udp/]*

5.3 Tailoring Policies

Tailoring is not a requirement to bring the document into compliance with the standards set by IEEE 29148-2018. Tailoring should only occur when conformance to the standard is not possible or practical. The act of tailoring is the modification and/or removal of one of the content sections outlined in this document, adding additional information items for organization is not considered tailoring. Tailoring should only occur when factors or circumstances:

- surround an organization that is using the document
- influence a project using this document to meet an agreement
- reflect the needs of an organization.

When tailoring the document, the following activities shall be implemented:

- Identify and document the circumstances that may influence tailoring.
    - novelty, size and complexity
    - stability of operating environments
    - variety in operating environments
    - starting date and duration
    - emerging technology
    - availability of services of enabling systems

    - other standards with which the document needs to conform.
- Identify and get input from all parties impacted by the tailoring process.
    - Such as stakeholders, contributors, and other interested parties
- Delete the information contents that require tailoring.

6. Appendix B – Copyright

This document is based on a template meeting the ISO/IEC/IEEE 29148-2018 standard, available at https://www.iso.org/standard/72089.html. Template authors are:

Dr. rer. nat. Bastian Tenbergen,
Associate Professor of Software Engineering
bastian@tenbergen.org

Mikayla Conner-Spagnola, MA
Independent Consultant
mconner@oswego.edu

Department of Computer Science
State University of New York at Oswego
Oswego, NY 13126, United States

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License. For more information, please see http://creativecommons.org/licenses/by-sa/4.0/